

TestHub 智能测试管理平台

基于 AI 驱动的全栈测试管理平台

Python 3.12 Django 4.2 Vue 3.3 License MIT

项目简介

TestHub 是一个功能强大的智能测试管理平台，集成了 AI 需求分析、测试用例管理、API 测试、UI 自动化测试 等多个模块，旨在提升测试效率和质量。平台采用 Django + Vue3 技术栈，提供现代化的用户界面和丰富的功能特性。

核心特性

AI 智能化能力

- AI 需求分析: 自动解析需求文档（PDF/Word/TXT），智能提取业务需求
- 智能测试用例生成: 基于需求自动生成测试用例，支持多种测试类型
- 智能助手: 集成 Dify AI 助手，提供测试咨询和问题解答
- 多模型支持: 支持 DeepSeek、通义千问、硅基流动等多种 AI 模型
- AI 智能模式: 基于 Browser-use 的智能浏览器自动化，AI 理解页面并自动完成测试

安全机制

- JWT 认证: 采用企业级 JWT 双 Token 安全机制
- 自动刷新: Access Token 过期前自动刷新，无感续期
- Token 黑名单: 登出时自动将 Token 加入黑名单，防止重放攻击
- 请求队列: Token 刷新期间请求自动排队等待，确保请求不丢失

统一配置中心

- 环境检测: 自动检测系统浏览器和 Playwright 环境
- 驱动管理: 一键安装和更新浏览器驱动
- AI 模型配置: 统一管理多种 AI 模型的 API 配置
- 连接测试: 支持 AI 模型连接测试和验证

测试用例管理

- 完整的用例生命周期管理: 创建、编辑、版本控制、归档
- 灵活的用例组织: 支持项目、版本、标签等多维度分类
- 详细的用例步骤: 支持步骤化用例设计，包含前置条件、操作步骤、预期结果
- 附件和评论: 支持用例附件上传和团队协作评论

测试用例评审

- 评审流程管理: 支持多人评审、评审模板、检查清单
- 评审状态跟踪: 待评审、评审中、已通过、已拒绝等状态管理
- 评审意见记录: 支持整体意见、用例意见、步骤意见等多层级反馈
- 评审模板: 可自定义评审检查清单和默认评审人

API 测试

- 项目和集合管理: 支持 HTTP/WebSocket 协议，树形结构组织 API
- 请求管理: 支持 GET/POST/PUT/DELETE/PATCH 等多种 HTTP 方法
- 环境变量: 全局和局部环境变量管理，支持变量替换
- 测试套件: 批量执行 API 请求，支持断言和执行顺序配置
- 请求历史: 完整的请求执行历史记录和结果追踪
- 定时任务: 支持定时执行测试套件，邮件/Webhook 通知
- 测试报告: 自动生成 Allure 测试报告

UI 自动化测试（Web）

- 双引擎支持: 支持 Selenium 和 Playwright 两种自动化引擎
- 元素管理: 元素库管理，支持多种定位策略（ID、XPath、CSS 等）
- 页面对象模式: 支持 POM 设计模式，提高脚本可维护性
- 测试脚本: 可视化脚本编辑器，支持步骤录制和回放
- 测试套件: 批量执行测试脚本，支持多浏览器（Chrome/Firefox/Edge）
- 执行记录: 详细的执行日志、截图、视频录制
- 定时任务: 支持 Cron 表达式、固定间隔、单次执行
- AI 智能模式:
 - 基于 Browser-use 框架的智能浏览器自动化
 - AI 理解页面结构并自动完成测试任务
 - 支持文本模式（基于 DOM 解析）和视觉模式（基于截图识别）
 - 支持多种 AI 模型：OpenAI、Anthropic、Google Gemini、DeepSeek、硅基流动等
 - 智能任务规划和步骤自动生成

📱 APP 自动化测试（Android）新增 📌【已完整实现】

- **Airtest 框架**: 基于图像识别的 Android APP 自动化测试
- **设备管理**: 支持本地模拟器和远程设备，设备资源池管理
- **设备锁定**: 多用户环境下的设备锁定机制，避免资源冲突
- **ADB 集成**: 自动发现设备、连接远程设备、设备信息查询
- **元素管理**: 支持图片元素、坐标元素、区域元素三种定位方式
- **多分辨率适配**: 不同分辨率下的元素配置管理
- **组件化编排**: 基础组件定义、自定义组件组合、组件包导入导出
- **UI Flow**: JSON 格式的 UI 流程编排，支持10+ Airtest动作
- **变量管理**: 支持 global/local/outputs 作用域，{{variable}} 语法
- **测试执行**: Celery异步执行 + pytest + Allure 报告生成
- **执行引擎**: AirtestBase + UiFlowRunner + AppTestExecutor 完整实现
- **进度追踪**: 实时执行进度、步骤统计、通过率计算
- **使用统计**: 元素使用次数追踪，优化元素管理
- **API 完整**: 40+ RESTful API 接口，支持所有功能操作
- **前端页面**: 7个完整页面（Dashboard/设备/元素/用例/执行记录）
- **代码编辑器**: Monaco Editor 集成，支持 JSON 语法高亮
- **图片上传**: 支持元素图片拖拽上传和预览
- **实时更新**: 执行记录自动刷新，实时进度展示

📊 测试执行与报告

- **测试计划**: 创建测试计划，关联项目、版本和测试用例
- **测试执行**: 手动和自动化测试执行，实时记录测试结果
- **执行历史**: 完整的执行历史追踪和结果对比
- **测试报告**: 多维度数据统计和可视化图表
- **Allure 集成**: 支持生成专业的 Allure 测试报告

🏭 数据工厂

- **字符工具**（9个功能）: 字符串处理、文本对比、正则表达式测试、字数统计、大小写转换
- **编码工具**（12个功能）: Base64编解码、时间戳转换、Unicode转换、进制转换、颜色值转换、URL编解码、JWT解码、条形码/二维码生成、图片Base64转换
- **随机工具**（6个功能）: 随机数、随机字符串、UUID、随机布尔值、随机列表元素
- **加密工具**（8个功能）: MD5/SHA1/SHA256/SHA512哈希、AES加密解密、HMAC签名
- **测试数据**（4个功能）: 中文姓名、手机号、邮箱、地址生成
- **JSON工具**（8个功能）: JSON格式化（树形展示）、JSON压缩、JSON校验、JSONPath查询、JSON对比、JSON转XML/YAML/CSV
- **Crontab工具**（4个功能）: 生成/解析Crontab表达式、获取下次执行时间、验证表达式
- **标签系统**: 支持多标签管理，可在接口测试和UI测试中引用带标签的数据
- **使用记录**: 工具使用历史记录和统计
- **场景筛选**: 按使用场景（数据生成、格式转换、数据验证、加密解密）筛选工具
- **数据引用**: 在接口测试（请求参数、断言、前置条件）和UI测试（测试步骤、输入数据、断言）中引用数据工厂数据

👥 项目与团队管理

- **项目管理**: 多项目支持，项目成员和角色管理
- **版本管理**: 版本规划和测试用例关联
- **权限控制**: 基于项目的成员角色权限管理
- **用户配置**: 个性化用户设置和偏好配置

🏗️ 技术架构

后端技术栈

- **框架**: Django 4.2 + Django REST Framework
- **数据库**: MySQL 8.0+ (PyMySQL)
- **API 文档**: drf-spectacular (Swagger/ReDoc)
- **安全认证**: JWT (rest_framework_simplejwt) + Token 黑名单
- **AI 集成**:
 - browser-use: AI 驱动的浏览器自动化
 - langchain-openai: LLM 集成框架
 - 多模型支持：OpenAI、Anthropic、Google Gemini、DeepSeek、硅基流动等
- **自动化测试**: Selenium, Playwright, Allure
- **HTTP 客户端**: httpx (异步 HTTP)
- **定时任务**: Django APScheduler

前端技术栈

- **框架**: Vue 3.3 + Composition API
- **构建工具**: Vite 4.4
- **UI 组件**: Element Plus 2.3
- **状态管理**: Pinia 2.1
- **路由**: Vue Router 4.2
- **HTTP 客户端**: Axios 1.5
- **数据可视化**: ECharts 5.4
- **代码编辑器**: Monaco Editor
- **其他**: vuedraggable (拖拽), xlsx (Excel), dayjs (日期)

📁 项目结构

```

testhub_platform/
├─ apps/                                # Django 应用模块
│   └─ users/                          # 用户管理
│   └─ projects/                       # 项目管理
│   └─ testcases/                      # 测试用例管理
│   └─ testsuites/                    # 测试套件管理
│   └─ executions/                    # 测试执行管理
│   └─ data_factory/                  # 数据工厂
│   └─ reports/                      # 测试报告
│   └─ reviews/                      # 用例评审管理
│   └─ versions/                     # 版本管理
│   └─ core/                          # 核心功能模块
│       └─ models.py                  # 统一通知配置模型
│       └─ views.py                   # 核心功能视图
│       └─ management/commands/      # 管理命令
│           └─ run_all_scheduled_tasks.py # 统一定时任务调度器
│           └─ init_locator_strategies.py # 初始化元素定位策略
│           └─ download_webdrivers.py   # 下载浏览器驱动
│   └─ requirement_analysis/          # AI 需求分析
│   └─ assistant/                    # 智能助手
│   └─ api_testing/                  # API 测试
│   └─ ui_automation/                # UI 自动化测试
├─ backend/                          # Django 项目配置
│   └─ settings.py                   # 项目设置
│   └─ urls.py                       # URL 路由
│   └─ middleware.py                 # 中间件
├─ frontend/                          # Vue3 前端
│   └─ src/
│       └─ api/                      # API 接口
│       └─ components/               # 公共组件
│       └─ views/                    # 页面视图
│           └─ auth/                 # 登录注册
│           └─ projects/              # 项目管理
│           └─ testcases/             # 测试用例
│           └─ data-factory/          # 数据工厂
│           └─ reviews/              # 用例评审
│           └─ requirement-analysis/  # 需求分析
│           └─ assistant/             # 智能助手
│           └─ api-testing/           # API 测试
│           └─ ui-automation/        # UI 自动化
│               └─ ai/                # AI 智能模式
│               └─ config/            # 配置管理
│               └─ suites/            # 测试套件
│               └─ configuration/     # 统一配置中心
│       └─ stores/                   # Pinia 状态管理
│       └─ router/                   # 路由配置
│       └─ utils/                    # 工具函数
│       └─ assets/                   # 静态资源
│   └─ package.json
├─ media/                            # 媒体文件（上传文件、截图等）
├─ logs/                             # 日志文件
│   └─ scheduler.log                # 统一调度器日志
├─ allure/                           # Allure 测试报告
├─ requirements.txt                  # Python 依赖
└─ manage.py                         # Django 管理脚本

```

快速开始

环境要求

- **Python:** 推荐Python3.12,其他版本可能会存在兼容性问题
- **Node.js:** 18+(开发环境必须安装Node.js用于构建前端项目,生产可不安装)
- **MySQL:** 8.0+(必须安装MySQL客户端,用于执行数据库迁移等操作)
- **Java:** 17+ (可选,用于运行浏览器驱动、Allure 报告生成等,否则会生成报告失败)
- **Redis:** 6.0+ (可选,用于APP自动化测试相关)

- **浏览器驱动:** ChromeDriver / GeckoDriver (用于 UI 自动化,建议提前下载好)

后端部署

1. 克隆项目

```
git clone <repository-url>
cd testhub_platform
```

2. 创建虚拟环境

```
python -m venv venv
# Windows
venv\Scripts\activate
# Linux/Mac
source venv/bin/activate
```

3. 安装依赖

```
pip install -r requirements.txt
```

4. 配置环境变量

```
# 复制示例配置文件到 .env 文件
# 按照 .env文件模板配置你的数据库连接信息等
cp .env.example .env
```

5. 初始化数据库

```
# 创建数据库
mysql -u root -p
CREATE DATABASE testhub CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
EXIT;

# 创建 migrations 目录 (如果不存在)
mkdir -p apps/testcases/migrations
echo "# This file is intentionally left empty" > apps/testcases/migrations/__init__.py

# 执行迁移
python manage.py makemigrations
python manage.py migrate

# 创建超级用户
python manage.py createsuperuser
```

6. 初始化UI自动化测试定位策略

```
# 根目录执行
python manage.py init_locator_strategies
```

7. 初始化app自动化组件库

```
# 根目录执行
python manage.py load_component_pack
```

8. 启动定时任务

```
# 启动统一任务调度器(同时管理API和UI模块)
python manage.py run_all_scheduled_tasks
```

9. 启动服务

```
# 启动 Django 开发服务器
python manage.py runserver
```

10. 启动Celery服务

```
# 启动 Celery 开发服务(可选，用于处理APP自动化任务)
celery -A backend worker -l info
```

数据工厂模块初始化

数据工厂模块需要创建数据库表：

```
# 创建数据工厂表
python manage.py makemigrations data_factory
python manage.py migrate data_factory
```

详细使用说明：请查看 [数据工厂使用说明.md](#) 获取完整的功能介绍、使用技巧和最佳实践。

快速开始指南：请查看 [数据工厂快速开始.md](#) 快速上手数据工厂功能。

前端部署

1. 安装依赖

```
cd frontend
npm install
```

2. 启动开发服务器

```
npm run dev
```

3. 构建生产版本

```
npm run build
```

访问应用

- 前端: <http://localhost:3000>
- 后端 API: <http://localhost:8000>
- API 文档: <http://localhost:8000/api/docs/>
- Admin 后台: <http://localhost:8000/admin/>

文档

- [更新日志 \(CHANGELOG\)](#): 查看版本更新历史和重要变更
- [数据工厂使用说明](#): 数据工厂 功能完整介绍和使用技巧
- [数据工厂快速开始](#): 数据工厂 快速上手指南
- [数据工厂功能说明](#): 数据工厂 功能详细说明
- [数据工厂API接口文档](#): 数据工厂 API 接口文档
- [UI自动化测试执行说明](#): UI 自动化测试执行指南
- [WebDriver驱动管理优化说明](#): WebDriver 驱动管理优化说明
- [用例评审管理功能说明](#): 用例评审管理功能说明
- [问题排查指南](#): 常见问题排查指南

核心功能模块说明

1. 核心功能模块 (core)

概述: core 模块是跨模块的通用功能模块，提供全局共享的管理命令和统一配置管理。

管理命令:

- `run_all_scheduled_tasks`: 统一定时任务调度器
 - 同时调度 API 测试和 UI 自动化模块的定时任务
 - 支持自定义检查间隔（默认60秒）
 - 支持单次执行模式（`--once`）
 - 详细日志输出，便于调试和监控
- `init_locator_strategies`: 初始化UI自动化元素定位策略
 - 创建/更新12种常用元素定位策略
 - 通用策略：ID, CSS, XPath, name, class, tag
 - Playwright 专用策略：text, placeholder, role, label, title, test-id

- `download_webdrivers`: 下载浏览器驱动
 - 支持 Chrome (ChromeDriver)
 - 支持 Firefox (GeckoDriver)
 - 支持 Edge (EdgeDriver)
 - 自动缓存, 后续使用更快

数据模型:

- `UnifiedNotificationConfig`: 统一通知配置
 - 支持企业微信、钉钉、飞书等多种 Webhook 机器人
 - 每个机器人可独立配置启用状态
 - 支持 API 测试和 UI 自动化测试模块独立开关
 - JSON 格式存储多个机器人配置

API 路由:

- `/api/core/notification-configs/`: 统一通知配置管理

日志文件:

- `logs/scheduler.log`: 统一调度器运行日志

2. AI 需求分析模块 (`requirement_analysis`)

功能:

- 上传需求文档 (PDF/Word/TXT)
- AI 自动解析需求文档内容
- 提取业务需求和功能点
- 基于需求自动生成测试用例
- 支持多种 AI 模型配置

数据模型:

- `RequirementDocument`: 需求文档
- `RequirementAnalysis`: 需求分析记录
- `BusinessRequirement`: 业务需求
- `GeneratedTestCase`: 生成的测试用例
- `AnalysisTask`: 分析任务
- `AIModelConfig`: AI 模型配置

3. 智能助手模块 (`assistant`)

功能:

- 集成 Dify AI 助手
- 多会话管理
- 聊天历史记录
- 测试咨询和问题解答

数据模型:

- `DifyConfig`: Dify API 配置
- `AssistantSession`: 助手会话
- `ChatMessage`: 聊天消息

4. API 测试模块 (`api_testing`)

功能:

- API 项目和集合管理
- HTTP/WebSocket 请求管理
- 环境变量管理
- 测试套件和自动化执行
- 请求历史和结果追踪
- 定时任务和通知
- Allure 报告生成

数据模型:

- `ApiProject`: API 项目
- `ApiCollection`: API 集合
- `ApiRequest`: API 请求
- `Environment`: 环境变量
- `TestSuite`: 测试套件
- `RequestHistory`: 请求历史
- `ApiScheduledTask`: 定时任务
- `ApiNotificationConfig`: 通知配置

4.5. 数据工厂模块 (`data_factory`)

功能:

- **字符工具** (9个功能): 去除空格换行、字符串替换、转义反转义、字数统计、文本对比、正则测试、大小写转换、字符串格式化
- **编码工具** (12个功能): 生成条形码/二维码、时间戳转换、进制转换、Unicode/ASCII转换、颜色值转换、Base64编解码、URL编解码、JWT解码、图片Base64转换
- **随机工具** (6个功能): 随机整数/浮点数、随机字符串、UUID生成、随机布尔值、随机列表元素
- **加密工具** (8个功能): MD5/SHA1/SHA256/SHA512哈希、AES加密解密、HMAC签名
- **测试数据** (4个功能): 生成中文姓名、中国手机号、中国邮箱、中国地址
- **JSON工具** (8个功能): JSON格式化 (树形展示)、JSON压缩、JSON校验、JSONPath查询、JSON对比、JSON转XML/YAML/CSV
- **Crontab工具** (4个功能): 生成/解析Crontab表达式、获取下次执行时间、验证表达式
- **标签系统**: 支持多标签管理, 可在接口测试和UI测试中引用带标签的数据
- **使用记录**: 工具使用历史记录和统计
- **场景筛选**: 按使用场景 (数据生成、格式转换、数据验证、加密解密) 筛选工具
- **数据引用**: 在接口测试 (请求参数、断言、前置条件) 和UI测试 (测试步骤、输入数据、断言) 中引用数据工厂数据

核心特性:

- **51个实用工具**: 覆盖字符处理、编码转换、随机数据、加密解密、测试数据、JSON处理、Crontab管理等多个场景
- **标签管理**: 每条数据记录可添加多个标签, 支持按标签筛选和管理
- **数据引用**: 在接口测试和UI测试中通过DataFactorySelector组件引用带标签的数据
- **历史记录**: 完整的工具使用历史, 支持按工具分类、工具名称、标签等多维度查询
- **实时预览**: JSON格式化支持树形展示、展开/折叠、实时预览 (300ms防抖)
- **状态持久化**: JSON格式化的展开/折叠状态自动保存到localStorage

数据模型:

- DataFactoryRecord: 数据工厂使用记录
 - tool_name: 工具名称
 - tool_category: 工具分类 (string/encoding/random/encryption/test_data/json/crontab)
 - tool_scenario: 使用场景 (data_generate/format_convert/data_validation/encrypt)
 - input_data: 输入数据 (JSON)
 - output_data: 输出数据 (JSON)
 - is_saved: 是否保存
 - tags: 标签 (JSON数组)
 - created_at: 创建时间
 - updated_at: 更新时间

API 路由:

- /api/data-factory/: 数据工厂记录管理 (CRUD)
- /api/data-factory/execute/: 执行工具
- /api/data-factory/download_static_file/{filename}/: 下载生成的文件 (条形码、二维码等)

详细使用说明: 请查看 [数据工厂使用说明.md](#) 获取完整的功能介绍、使用技巧和最佳实践。

5. UI 自动化测试模块 (ui_automation)

功能:

- 元素库管理 (支持多种定位策略)
- 页面对象模式 (POM)
- 测试脚本编辑和执行
- 测试套件批量执行
- 多浏览器支持
- 执行截图和视频录制
- 定时任务调度
- **AI 智能测试模式**:
 - 基于 Browser-use 框架的智能浏览器自动化
 - AI 自动理解页面结构并生成测试步骤
 - 支持文本模式 (基于 DOM 解析) 和视觉模式 (基于截图识别)
 - 智能任务规划和执行
 - 执行过程实时日志记录

核心组件:

- ai_base.py: Browser-use 基础框架和补丁
- ai_agent.py: AI Agent 实现 (BrowserAgent 类)
- ai_models.py: 多 AI 模型统一接口

数据模型:

- UiProject: UI 项目
- Element: 元素
- ElementGroup: 元素分组
- PageObject: 页面对象
- TestScript: 测试脚本
- TestCase: 测试用例
- TestSuite: 测试套件
- TestExecution: 测试执行
- UiScheduledTask: 定时任务
- AICase: AI 智能用例
- AIIntelligentModeConfig: AI 智能模式配置

6. 统一配置中心模块 (configuration)

功能:

- **环境检测:** 自动检测系统已安装的浏览器
- **驱动管理:** 一键安装 Playwright 浏览器驱动
- **AI 模型配置:**
 - 支持多种 AI 提供商：通义千问、DeepSeek、硅基流动、本地模型
 - 按角色配置：测试用例编写器、测试用例评审员、Browser Use 文本模式
 - API 密钥、基础 URL、模型名称、参数配置
 - 连接测试功能

API 路由:

- `/api/ui-automation/config/environment/`: 环境配置
- `/api/ui-automation/config/ai-mode/`: AI 智能模式配置

7. 测试用例评审模块 (reviews)

功能:

- 创建评审任务
- 分配评审人员
- 评审意见记录
- 评审模板管理
- 评审状态跟踪

数据模型:

- `TestCaseReview`: 测试用例评审
- `ReviewAssignment`: 评审分配
- `TestCaseReviewComment`: 评审意见
- `ReviewTemplate`: 评审模板

8. 测试执行模块 (executions)

功能:

- 测试计划管理
- 测试执行记录
- 执行历史追踪
- 执行结果统计

数据模型:

- `TestPlan`: 测试计划
- `TestRun`: 测试执行
- `TestRunCase`: 测试执行用例
- `TestRunCaseHistory`: 执行历史

配置说明

JWT 安全配置

项目采用企业级 JWT 双 Token 安全机制：

后端配置 (backend/settings.py):

```
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30), # Access Token 30分钟
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7), # Refresh Token 7天
    'ROTATE_REFRESH_TOKENS': True, # 刷新时轮换 Refresh Token
    'BLACKLIST_AFTER_ROTATION': True, # 旧 Refresh Token 加入黑名单
    'UPDATE_LAST_LOGIN': True,
    'ALGORITHM': 'HS256',
    'AUTH_HEADER_TYPES': ('Bearer',),
}
```

安全特性:

- 双 Token 机制：短期 Access Token + 长期 Refresh Token
- 自动刷新：Token 过期前 5 分钟自动刷新，无感续期
- Token 黑名单：登出时将 Refresh Token 加入黑名单，防止重放攻击
- 请求队列：Token 刷新期间的请求自动排队等待
- 防循环机制：logout 函数包含防循环调用保护

前端 Token 管理:

- Token 存储在 localStorage
- 请求拦截器自动添加 Bearer Token
- 响应拦截器处理 401 错误并自动刷新 Token

AI 智能模式配置

在统一配置中心可以配置多种 AI 模型：

支持的 AI 提供商：

- **OpenAI**: GPT-4、GPT-3.5 等模型
- **Azure OpenAI**: Azure 托管的 OpenAI 服务
- **Anthropic**: Claude 系列模型
- **Google Gemini**: Gemini Pro、Gemini Flash
- **DeepSeek**: DeepSeek 系列模型
- **硅基流动**: 聚合多种 AI 模型

配置角色：

- `testcase_writer`: 测试用例编写
- `testcase_reviewer`: 测试用例评审
- `browser_use_text`: Browser Use 文本模式（DOM 解析）
- `browser_use_vision`: Browser Use 视觉模式（截图识别）- 暂未实现

配置参数：

- API Key: API 访问密钥
- Base URL: API 端点地址（可选）
- Model Name: 模型名称
- Temperature: 温度参数（控制随机性）
- Max Tokens: 最大生成 Token 数

连接测试: 配置完成后可使用“测试连接”功能验证配置是否正确。

AI 需求分析配置

在系统配置中心可以配置多种 AI 模型：

- **DeepSeek**: 用于需求分析和用例生成
- **通义千问**: 备选 AI 模型
- **硅基流动**: 备选 AI 模型
- **自定义模型**: 支持配置自定义 API

Dify 助手配置

配置 Dify API 以启用智能助手功能：

- API URL: Dify API 端点
- API Key: Dify API 密钥

UI 自动化配置

- **执行引擎**: Selenium / Playwright
- **浏览器**: Chrome / Firefox / Edge
- **WebDriver**: 自动下载或手动配置驱动路径
- **运行模式**: 有头模式 / 无头模式
- **AI 智能模式**:
 - 文本模式：基于 DOM 解析，快速高效
 - 视觉模式：基于截图识别，适合复杂页面

通知配置

- **邮件通知**: SMTP 配置
- **Webhook 通知**: 企业微信、钉钉等

🗄️ 数据库设计

项目使用 MySQL 数据库，主要表结构包括：

- **用户相关**: `users`, `user_profiles`
- **项目管理**: `projects`, `project_members`, `versions`
- **测试用例**: `testcases`, `testcase_steps`, `testcase_attachments`, `testcase_comments`
- **测试套件**: `testsuites`, `testsuite_cases`
- **测试执行**: `test_plans`, `test_runs`, `test_run_cases`
- **用例评审**: `testcase_reviews`, `review_assignments`, `review_comments`
- **核心配置**: `core_unifiednotificationconfig` - 统一通知配置
- **需求分析**: `requirement_documents`, `requirement_analyses`, `business_requirements`, `generated_test_cases`
- **AI 配置**: `ai_model_configs`, `prompt_configs` - AI 模型和提示词配置
- **智能助手**: `dify_configs`, `assistant_sessions`, `chat_messages`
- **API 测试**: `api_projects`, `api_collections`, `api_requests`, `api_environments`, `test_suites`, `request_history`, `api_scheduled_tasks`
- **UI 自动化**: `ui_projects`, `ui_elements`, `element_groups`, `ui_page_objects`, `ui_test_scripts`, `ui_test_cases`, `ui_test_suites`, `ui_test_executions`, `ui_scheduled_tasks`, `ai_cases`, `ai_intelligent_mode_configs`
- **数据工厂**: `data_factory_record` - 工具使用记录表
- **JWT 安全**: `blacklisted_token`, `outstanding_token` - Token 黑名单管理

📖 贡献指南

欢迎提交 Issue 和 Pull Request 来帮助改进项目！

1. Fork 本仓库
2. 创建特性分支 (`git checkout -b feature/AmazingFeature`)
3. 提交更改 (`git commit -m 'Add some AmazingFeature'`)
4. 推送到分支 (`git push origin feature/AmazingFeature`)
5. 开启 Pull Request

📄 许可证

本项目采用 MIT 许可证 - 详见 [LICENSE](#) 文件

📞 联系方式

如有问题或建议，欢迎通过 Issue 反馈。
